

CoRE Stack Village Analytics Tool (CSVAT)

Software Requirements Specification (SRS)

Aaman Sheikh — M Rayhan Khan — Saksham Kushwaha

1. Introduction

1.1 Purpose

The purpose of this document is to specify the functional and non-functional requirements for the **CoRE Stack Village Analytics Tool (CSVAT)**. CSVAT is a **web analytics product** that enables practitioners, researchers, and planners to generate village-level socio-ecological analytics from CoRE Stack datasets through a browser-based interface, without requiring a local programming environment.

1.2 Problem Statements

We chose Data Exploration and Insights problems [Bring boundary flexibility] to start working with:

- 1. CoRE stack provides analytics at the micro-watershed level by default but users often want the same stats at the village level computed from the underlying rasters. For a given village polygon, using the underlying LULC rasters over the years, compute and visualize how cropping intensity (single/double/triple) has changed over the years.
- 2. For the same village, compute seasonal surface-water availability (perennial, winter, monsoon) across years and show trends in area (hectares) per season.
- 3. Identify areas within a village that lost tree cover over a chosen period and estimate hectares of degraded land.

1.3 Scope

CSVAT provides a web-based workflow that allows users to:

- Select a village using administrative identifiers or upload a custom village polygon.
- Resolve village boundaries using CoRE Stack and Google Earth Engine (GEE) assets.
- Compute village-level socio-ecological indicators from CoRE Stack raster layers.
- Generate interactive HTML reports and downloadable static PDF reports.

The system supports both **server-side** and **client-side** execution of the analytics pipeline. Users or deployment configurations may select the execution mode based on performance, resource availability, or operational constraints.

Based on observed performance, scalability, and maintainability, the system may standardize on a single execution mode in future iterations.

2. Overall Description

2.1 Product Perspective

CSVAT is a browser-accessible analytics layer built on top of the CoRE Stack ecosystem. It enables dynamic aggregation of raster-based analytics from hydrological units to village-level administrative boundaries through a web-based user interface.

The system supports two execution modes for the analytics pipeline:

- **Server-side execution**, where all analytical steps are performed on a backend service and rendered outputs (interactive HTML and PDF) are returned to the client.
- **Client-side execution**, where the analytics pipeline is executed within the user's browser, using WebAssembly (WASM) for computation and local generation of outputs.

The execution mode may be selected by the user or determined by deployment configuration. Both modes follow the same logical analytical pipeline and access data through existing CoRE Stack APIs.

Based on empirical performance evaluation and operational considerations, future versions of CSVAT may converge to a single preferred execution mode.

3. System Architecture & Data Flow

3.1 System Architecture

Architectural Rationale CSVAT follows a web-oriented client-server architecture in which the CoRE Stack infrastructure handles authentication and data provisioning. Village-specific analytics are executed either on the server or within the client browser, depending on the selected execution mode.

This flexible architecture allows performance and scalability trade-offs to be evaluated before standardizing on a single execution strategy.

Boundary Flexibility By clipping raster data using village polygons resolved from administrative inputs or uploaded boundaries, CSVAT decouples analytical units from the default micro-watershed boundaries used by the CoRE Stack.

3.2 User Flow and Data Processing Pipeline

Overview The system follows a deterministic pipeline initiated by a user-selected village boundary:

- User provides a village, tehsil, or district name, or uploads a GeoJSON boundary.
- The analytics pipeline is executed either on the server or on the client based on the selected execution mode, with identical outputs generated in both cases.
- The system resolves the village polygon and identifies the corresponding state, district, and tehsil.
- Data availability is verified for the identified tehsil.
- Relevant raster layers are retrieved from the CoRE Stack.
- Analytics are computed within the village boundary.
- An interactive HTML report is generated, with optional PDF export.

Error Handling If the village does not intersect an active tehsil, the pipeline terminates with an explicit error message and no partial outputs are generated.

4. Data Management

4.1 Input Data

- Administrative identifiers (village, tehsil, district names).
- Village boundary polygons provided as GeoJSON.

All spatial inputs are validated for format and spatial correctness.

4.2 Intermediate Data

Tehsil-level raster layers retrieved from the CoRE Stack are treated as read-only and used temporarily during computation. Intermediate data is not persisted beyond execution.

4.3 Output Data

Final outputs include:

- Interactive HTML reports.

- Downloadable static PDF reports.
- CSV files containing annual village-level statistics.

Outputs are deterministically generated from the same inputs.

5. User Personas

5.1 Persona 1: Rural Development Practitioner

Name: Meera Sharma

Role: Field Coordinator, Watershed Development Program

Technical Proficiency: Moderate (uses web tools, not programming environments)

Goals:

- Understand changes in cropping patterns and water availability at the village level.
- Identify areas of land degradation for planning interventions.
- Generate reports that can be shared with village councils and district officials.

Needs:

- Simple boundary selection through village name input.
- Clear visualizations without requiring GIS expertise.
- Downloadable PDF reports for offline sharing.

Pain Points:

- Existing analytics are available only at micro-watershed level.
- Difficulty aggregating raster data to administrative boundaries.
- Limited access to technical GIS tools.

5.2 Persona 2: Environmental Researcher

Name: Dr. Arjun Rao

Role: Academic Researcher in Socio-Ecological Systems

Technical Proficiency: High (familiar with GIS and programming)

Goals:

- Analyze long-term trends in vegetation cover and cropping intensity.

- Export structured datasets for further statistical analysis.
- Validate hypotheses using reproducible analytics.

Needs:

- Access to raw CSV outputs.
- Transparent analytical methodology.
- Deterministic and reproducible results.

Pain Points:

- Manual boundary clipping workflows are time-consuming.
- Lack of standardized village-level aggregation.

5.3 Persona 3: District Planning Officer

Name: Rakesh Singh

Role: District Planning Authority

Technical Proficiency: Low to Moderate

Goals:

- Quickly assess village-level environmental trends.
- Compare villages within a district.
- Use visual summaries for decision-making meetings.

Needs:

- Interactive HTML reports accessible via browser.
- Clear summary charts and highlighted spatial hotspots.

Pain Points:

- Limited time for complex technical tools.
- Need for actionable insights rather than raw data.

6. System Features & Functional Requirements

6.1 Boundary Acquisition and Validation

6.1.1 Functional Requirements

- **FR-BA-01:** The system shall accept a village boundary as a GeoJSON polygon.
- **FR-BA-02:** The system shall resolve village boundaries using administrative identifiers.
- **FR-BA-03:** The system shall validate that the boundary intersects at least one active tehsil.
- **FR-BA-04:** The system shall terminate execution with an appropriate error message if validation fails.

6.2 Cropping Intensity Analytics

6.2.1 Functional Requirements

- **FR-CI-01:** Fetch LULC raster layers for all available years.
- **FR-CI-02:** Classify cropped pixels into single, double, and triple crop categories.
- **FR-CI-03:** Compute area (hectares) for each category within the village boundary.
- **FR-CI-04:** Generate temporal visualizations of cropping intensity trends.

6.3 Seasonal Surface Water Analysis

6.3.1 Functional Requirements

- **FR-SW-01:** Retrieve surface water datasets from the CoRE Stack.
- **FR-SW-02:** Classify water pixels into perennial and seasonal categories.
- **FR-SW-03:** Compute seasonal water-covered area in hectares.
- **FR-SW-04:** Generate temporal water availability trends.

6.4 Vegetation and Degradation Tracking

6.4.1 Functional Requirements

- **FR-VD-01:** Compare vegetation-related LULC classes across time periods.
- **FR-VD-02:** Compute tree cover loss or gain within the village boundary.

- **FR-VD-03:** Estimate degraded land area.
- **FR-VD-04:** Summarize degradation trends in tabular and visual formats.

7. Data Story Generation

CSVAT presents analytical results as structured **data stories**, where insights are organized into ordered narrative sections combining interactive maps, charts, and summary statistics. Data stories are rendered as interactive HTML and may be exported as static PDFs.

8. Technology Stack

8.1 Frontend

The frontend of CSVAT shall be implemented as a web-based user interface using **React JS**. The frontend shall be responsible for user interaction, boundary input, execution mode selection, and rendering interactive HTML reports.

8.2 Backend

The backend of CSVAT shall be implemented using **Python** and shall handle analytics execution when the server-side execution mode is selected. Backend services shall be exposed through RESTful APIs implemented using a lightweight Python web framework (**FastAPI**).

The backend shall integrate with the following components:

- **Google Earth Engine (GEE)** for accessing administrative boundaries and raster assets.
- **GDAL** for raster processing and geospatial transformations.
- **CoRE Stack APIs** for data access and authentication.

The backend shall be responsible for executing the analytics pipeline, generating interactive HTML and PDF outputs, and returning the rendered results to the frontend.

8.3 Database

Persistent spatial and non-spatial metadata shall be stored using **PostGIS**. The database may be used for caching resolved boundaries, execution metadata, and analytics results where applicable.

9. MVP / Acceptance Criteria

The Minimum Viable Product (MVP) shall satisfy the following:

- Authenticate with CoRE Stack APIs.
- The system shall support both server-side and client-side execution of the analytics pipeline, with the ability to select the execution mode.
- Accept village boundaries via administrative input or GeoJSON upload.
- Resolve village boundaries and identify tehsil context.
- Retrieve and clip at least one CoRE Stack raster layer.
- Compute and visualize cropping intensity trends.
- Generate an interactive HTML report.
- Provide downloadable CSV and PDF outputs.

Traceability Each acceptance criterion maps directly to the functional requirements defined in this document.

10. Task Prioritization

The following priorities define implementation order and MVP focus:

10.1 High Priority (Core Functionality)

5.1 Boundary Acquisition and Validation

- Village boundary input and validation.
- Administrative boundary resolution.
- Active tehsil verification.

5.2 Core Analytics Pipeline

- Retrieval and clipping of raster layers.
- Cropping intensity computation.
- Generation of village-level area statistics.
- Basic HTML report generation.

10.2 Medium Priority (Extended Features)

- Seasonal surface water analytics.
- Vegetation and degradation tracking.
- Data story section structuring.
- PDF export.
- Client-side (WASM) execution support.
- Database-backed caching.

11. Non-Functional Requirements

11.1 Performance

- Server-side execution shall complete village-level analytics within minutes or seconds for a standard village boundary.
- Client-side execution (if enabled) shall handle moderate spatial extents without browser crashes or excessive memory usage.

11.2 Reproducibility

- Identical inputs shall produce identical analytical outputs.
- The server-side environment shall be reproducible using documented installation steps or containerized deployment (e.g., Docker).

11.3 Usability

- The system shall provide clear error messages for invalid boundaries or unavailable datasets.
- Interactive reports shall be readable on standard desktop browsers.

11.4 Security

- API keys shall not be exposed in frontend source code.
- All communication with CoRE Stack APIs shall occur over HTTPS.

11.5 Maintainability

- Codebase shall be modular and documented.
- Analytical components shall be independently testable.

11.6 Portability

- The backend shall be deployable on standard Linux-based cloud environments.
- The frontend shall support modern web browsers (Chrome, Firefox, Edge).